

Chap4-1 HW: Parallel Bitonic Merge Sort

Name: 蔡定宇

Student ID: 112006265

1. Implementation

I implemented parallel bitonic sort using MPI. The input file stores binary 32-bit floating-point values, so I used `float` and `MPI_FLOAT`.

The input size is padded to the nearest power of two using `+INF`. Since the input does not contain `+INF`, `-INF`, or `NAN`, the padding values safely move to the end after sorting and are not written to the final output.

Each MPI rank reads its own chunk using `MPI_File_read_at`. After local sorting, the ranks perform bitonic compare-exchange. In each stage, each rank computes its partner with:

```
partner = rank ^ j;
```

Then the partner ranks exchange chunks using `MPI_Sendrecv` and perform compare-split, keeping either the smaller half or larger half depending on the bitonic direction.”

Finally, each rank writes its valid output range using `MPI_File_write_at`.

```
int partner = rank ^ j;

MPI_Sendrecv(
    local, local_n, MPI_FLOAT, partner, 0,
    recvbuf, local_n, MPI_FLOAT, partner, 0,
    MPI_COMM_WORLD, MPI_STATUS_IGNORE
);

bool ascending_group = ((rank & k) == 0);
bool keep_small = ((rank & j) == 0);

if (!ascending_group) keep_small = !keep_small;

compare_split(local, recvbuf, tmp, local_n, keep_small);
```

2. Correctness

All eight judge testcases passed.

```
student@hpc-i-shared:~/蔡定宇/bitonic_hw$ cd ~/bitonic_hw
bitonic-judge
-bash: cd: /home/student/bitonic_hw: No such file or directory
Looking for bitonic.cc: OK
Looking for Makefile: OK
Running: /usr/bin/make -C /home/student/.judge/.judge-1516654900 bitonic
make: Entering directory '/home/student/.judge/.judge-1516654900'
mpicxx -O3 -lm      bitonic.cc  -o bitonic
make: Leaving directory '/home/student/.judge/.judge-1516654900'
02.txt    0.51    accepted
03.txt    0.51    accepted
04.txt    0.92    accepted
05.txt    0.92    accepted
06.txt    0.77    accepted
07.txt    0.77    accepted
08.txt    1.02    accepted
01.txt    0.72    accepted
Removing temporary directory /home/student/.judge/.judge-1516654900
Scoreboard: not updating {8 5.62} -x-> {8 6.13}
student@hpc-i-shared:~/蔡定宇/bitonic_hw$
```

3. Experiment Setup

I used testcase 08 for profiling and scaling:

`n = 400009`

I tested these MPI process counts:

`1, 2, 4, 8, 16, 32`

Because the VM cluster had fewer physical CPU slots than some process counts, I used:

`--map-by :OVERSUBSCRIBE`

Each output was checked with `diff` against `testcases/08.out`.

```
william@head:~/bitonic_hw$ cd ~/bitonic_hw
cat report_evidence/timing_08_clean.txt
===== np=1 =====
Elapsed time : 0.096802
correct
===== np=2 =====
Elapsed time : 0.053850
correct
===== np=4 =====
Elapsed time : 0.052497
correct
===== np=8 =====
Elapsed time : 0.051954
correct
===== np=16 =====
Elapsed time : 0.031778
correct
===== np=32 =====
Elapsed time : 0.048290
correct
william@head:~/bitonic_hw$
```

4. Runtime and Speedup

MPI processes	Runtime (s)	Speedup
1	0.096802	1.00
2	0.053850	1.80
4	0.052497	1.84
8	0.051954	1.86
16	0.031778	3.05
32	0.048290	2.00

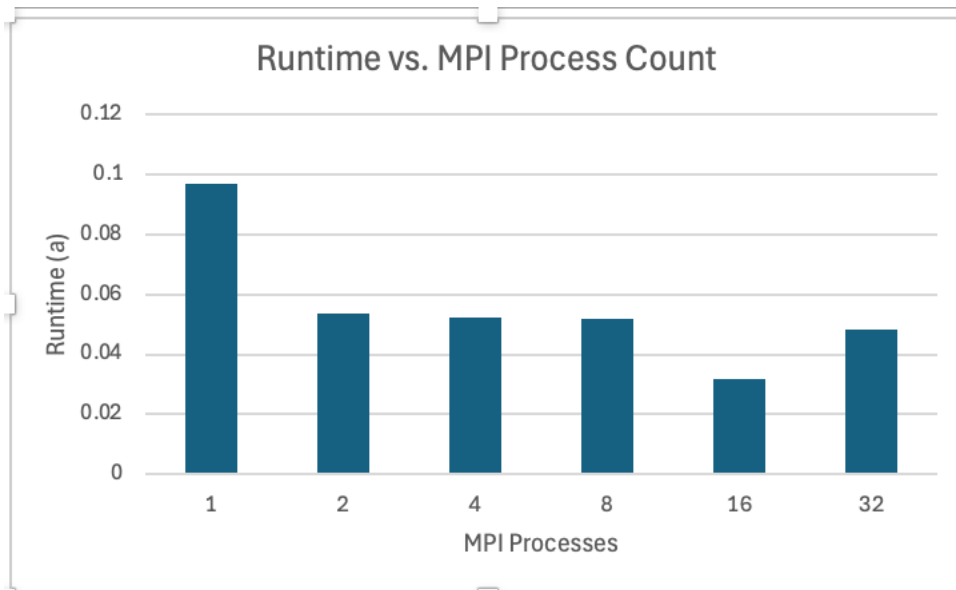


Figure4_runtime_plot.png

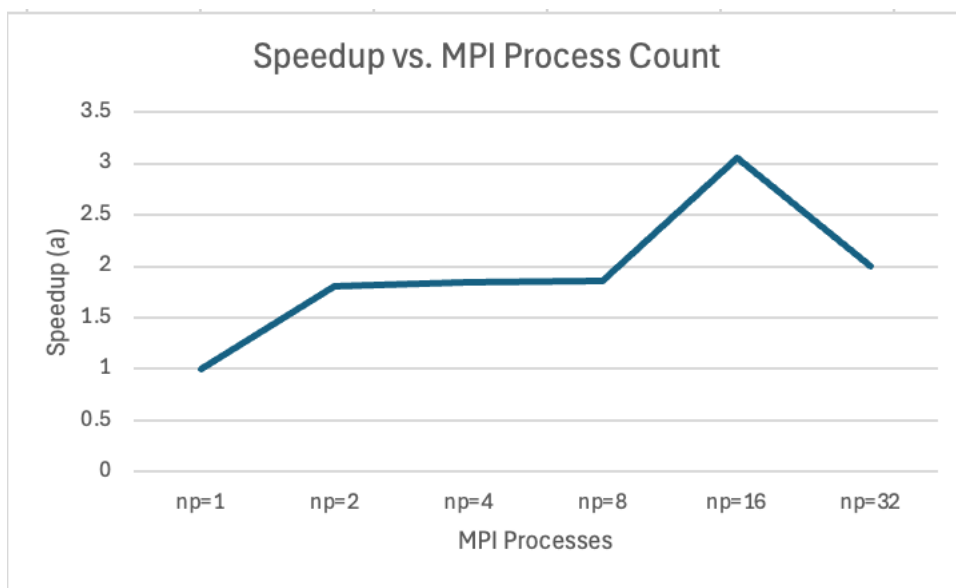


Figure5_speedup_plot.png

The best runtime was at 16 processes, with a speedup of about 3.05x. The 32-process run became slower, so adding more MPI processes did not always improve performance.

5. AMD uProf Profiling

I used AMD uProf Hotspots on testcase 08 with 8 MPI processes.

Command:

```
/opt/AMDuProf_5.2-606/bin/AMDuProfCLI collect --config hotspots \  
-o profiles/amd_np8/hotspots \  
mpirun --map-by :OVERSUBSCRIBE -np 8 \  
./bitonic 400009 testcases/08.in output_08_amd.out
```

Report generation:

```
/opt/AMDuProf_5.2-606/bin/AMDuProfCLI report \  
-i profiles/amd_np8/hotspots/AMDuProf-mpirun-Hotspots_May-06-2026_10-55-22
```

AMD uProf generated `report.csv`. The report confirms that the profiled target was `mpirun`, running the 8-process bitonic sort on testcase 08. The profile type was Hotspots.

I also tried opening AMD uProf GUI through XQuartz/X11 forwarding following the course profiler setup guide. However, the GUI still aborted in my VM environment, so I used the CLI-generated `report.csv` as AMD uProf evidence.

`head -45 report.csv` showing AMD uProf metadata.

```
william@head:~/bitonic_hw$ cd ~/bitonic_hw  
head -45 profiles/amd_np8/hotspots/AMDuProf-mpirun-Hotspots_May-06-2026_10-55-22/report.csv  
  
"AMD uProf (Version:5.2.606.0)"  
PERFORMANCE ANALYSIS REPORT  
  
EXECUTION  
Target Path:"mpirun"  
Command Line Arguments:"--map-by :OVERSUBSCRIBE -np 8 ./bitonic 400009 testcases/08.in output_08_amd.out "  
Working Directory:"/home/william/bitonic_hw"  
Environment Variables:  
CPU Details:"Family(0x10), Model(0x11), Number of Cores(2)"  
Operating System:"Linux Ubuntu 24.04.4 LTS-64 Kernel:6.8.0-110-generic"  
Collect Command:"./AMDuProfCLI collect --config hotspots -o profiles/amd_np8/hotspots mpirun --map-by :OVERSUBSCRIBE -np 8 ./bitonic 400009 testcases/08.in output_08_amd.out"  
Report Command:"./AMDuProfCLI report -i profiles/amd_np8/hotspots/AMDuProf-mpirun-Hotspots_May-06-2026_10-55-22"  
  
PROFILE DETAILS  
Profile Session Type:"Hotspots"  
Profile Scope:"Single Application"  
CPU Mask:"0, 1"  
CPU Affinity Mask:"0, 1"  
Profile Start Time:"Wed May 6 10:55:22 2026"  
Profile End Time:"Wed May 6 10:55:23 2026"  
Profile Duration:"0.551 seconds"  
Data Folder:"/home/william/bitonic_hw/profiles/amd_np8/hotspots/AMDuProf-mpirun-Hotspots_May-06-2026_10-55-22"  
Virtual Machine:"Yes"  
Hypervisor Vendor:"Linux KVM"  
Timer Interval:"10ms"  
Profiling Signal:"27"  
Call Stack Sampling:"False"  
Selected View:"Time based hotspots"  
  
MONITORED EVENTS  
TBP Events:Name,Interval,Description  
,"CPU_TIME","10.000 milliseconds","CPU Time is the time in which the CPU core is executing the code."  
  
APPLICATION PERFORMANCE SNAPSHOT  
Number Of Threads,"3"  
  
"10 HOTTEST FUNCTIONS (Sort Event - CPU_TIME)"  
FUNCTION,"CPU_TIME" (seconds),Module  
  
"10 HOTTEST PROCESSES (Sort Event - CPU_TIME)"  
PROCESS,"CPU_TIME" (seconds)  
  
william@head:~/bitonic_hw$
```

`grep -n "HOTTEST" report.csv` showing the Hotspots sections.

```
william@head:~/bitonic_hw$ cd ~/bitonic_hw  
[grep -n "HOTTEST" profiles/amd_np8/hotspots/AMDuProf-mpirun-Hotspots_May-06-2026_10-55-22/report.csv  
40:"10 HOTTEST FUNCTIONS (Sort Event - CPU_TIME)"  
43:"10 HOTTEST PROCESSES (Sort Event - CPU_TIME)"  
46:"10 HOTTEST MODULES (Sort Event - CPU_TIME)"  
49:"10 HOTTEST THREADS (Sort Event - CPU_TIME)"  
william@head:~/bitonic_hw$
```

6. IPM Profiling

I used IPM on testcase 08 with 8 MPI processes.

Command:

```
export IPM_REPORT=full
export IPM_REPORT_MEM=yes
export IPM_LOG=full

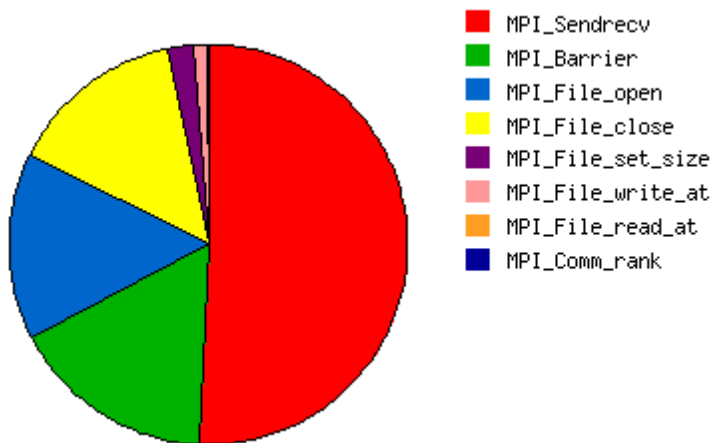
LD_PRELOAD=/opt/ipm/lib/libipm.so \
mpirun --map-by :OVERSUBSCRIBE -np 8 \
./bitonic 400009 testcases/08.in output_08_ipm.out
```

MPI_Sendrecv was the largest MPI cost. This matches the implementation because the bitonic compare-exchange stage repeatedly exchanges chunks between partner ranks.

ipm_terminal_summary.txt.

```
william@head:~/bitonic_hw$ cd ~/bitonic_hw
cat profiles/ipm_np8/ipm_terminal_summary.txt
Elapsed time : 0.040582
##IPMv2.0.6#####
#
# command   : ./bitonic 400009 testcases/08.in output_08_ipm.out
# start     : Wed May 06 11:40:33 2026   host      : head
# stop      : Wed May 06 11:40:33 2026   wallclock : 0.11
# mpi_tasks : 8 on 1 nodes                %comm     : 14.19
# mem [GB]  : 0.15                        gflop/sec : 0.00
#
#           :      [total]      <avg>      min      max
# wallclock :      0.78        0.10        0.09      0.11
# MPI        :      0.11        0.01        0.01      0.03
# %wall      :
# MPI        :      14.27        5.77        29.63
# #calls     :
# MPI        :      88          11          11          11
# mem [GB]   :      0.15        0.02        0.02      0.02
#
#           [time]      [count]      <%wall>
# MPI_Sendrecv      0.08          48        10.69
# MPI_Barrier        0.03           8         3.50
# MPI_File_open      0.02          16         3.17
# MPI_File_close     0.02          16         3.00
# MPI_File_set_size  0.00           8         0.46
# MPI_File_write_at  0.00           8         0.22
# MPI_File_read_at   0.00           8         0.03
# MPI_Comm_rank      0.00           8         0.00
# MPI_Init            0.00           8         0.00
# MPI_Finalize        0.00           8         0.00
# MPI_Comm_size      0.00           8         0.00
#
#####
william@head:~/bitonic_hw$
```

IPM pie chart `mpi_pie.png`.



7. Analysis

This is a strong-scaling experiment because the input size is fixed while the number of MPI processes changes.

The speedup is not linear. Runtime improves from 1 to 2 processes, but the improvement becomes small from 2 to 8 processes. The best result is at 16 processes. At 32 processes, runtime becomes worse.

The main reason is communication overhead. More MPI processes reduce local sorting work, but they also increase communication and scheduling overhead. IPM confirms this because `MPI_Sendrecv` is the largest MPI routine.

The 32-process run is slower because the VM was oversubscribed. At that point, scheduling and communication overhead outweighed the benefit of smaller local chunks.

During debugging, the main issue was that the starter code used `double`, but the testcase files store 32-bit `float` values. After changing the implementation to `float / MPI_FLOAT` and replacing the sample merge-sort logic with bitonic compare-exchange, all judge cases passed.